ProxiSpeak – Backend Development Work Report

Project: ProxiSpeak – Geospatial Proximity Audio Ecosystem
Role: Backend Developer / Backend Team Member
Technology Stack: Node.js, Express.js, Socket.IO, MongoDB, Mongoose, WebRTC Signaling, GitHub
Branch: `saipranav-backend`
Latest Commit: `dd13151` (`feat(backend): implement proximity audio engine, WebRTC signaling, office presence, and automated test suite`)
Status: 100% Completed & Verified

1. Executive Summary

All backend responsibilities for the ProxiSpeak Geospatial Proximity Audio Ecosystem have been fully engineered, verified, and pushed to the repository branch `saipranav-backend`.

The backend successfully connects the frontend application with the MongoDB database, synchronizes avatar movements in real-time, computes in-memory 2D Euclidean proximity with a configurable 100px earshot radius, enforces security gating on WebRTC audio signaling, and provides 27 automated test suites with zero failures.

2. Task-by-Task Audit: Requirements vs. Implementation

Task & Domain	Required Deliverables	What Was Implemented in Code	Status
Task 1: Backend Setup	Initialize Node.js & Express server, configure security headers, CORS, structure into clean modules, centralized error handling.	Express 5 initialized with Helmet security headers, permissive CORS, structured modules (`config` , `models` , `routes` , `controllers` , `services` , `sockets`), centralized `errorHandler.js` and `validateJson.js` .	**100% Complete** *(Verified)*
Task 2: MongoDB Integration	Connect to MongoDB Atlas, create Mongoose schemas (`User` , `Office` , `Position`), safe error handling, exclude credentials from git.	Live MongoDB Atlas connection verified. Mongoose models `User.js` , `Office.js` , `Position.js` integrated. `.env` added to `.gitignore` . Non-blocking debounced persistence via `positionSyncService.js` .	**100% Complete** *(Verified)*
Task 3: REST API Development	APIs for user registration/session, office information, and position data with proper status codes (200, 201, 400, 404, 415).	`POST & GET /api/users` , `POST & GET /api/offices` , `POST & GET /api/positions` , `GET /health` , `GET /` . Validated input payloads, strict JSON type checking, uniform error responses.	**100% Complete** *(Verified)*

Task & Domain	Required Deliverables	What Was Implemented in Code	Status
Task 4: Socket.IO Communication	Manage connection/disconnection, assign user session IDs, broadcast avatar moves, office-level scoping.	Modular Socket.io pipeline. `office:join` & `office:leave` manage room membership (`office:<id>`). `avatar:move` broadcasts `avatar:moved` to same office. Disconnect lifecycle management.	**100% Complete** *(Verified)*
Task 5: Proximity Detection	2D Euclidean distance calculation $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ d = (x2 – x1)2 + (y2 – y1)2 , 100px earshot threshold, emit proximity transitions.	In-memory `proximityService.js` computes exact pixel Euclidean distance. Dispatches `proximity:enter` when peers enter 100px and `proximity:leave` when peers move beyond 100px.	**100% Complete** *(Verified)*
Task 6: WebRTC Signaling	Exchange SDP offer, answer, and ICE candidates between peers. Strictly gate signaling so only eligible peers connect.	`webrtc:offer` , `webrtc:answer` , and `webrtc:ice-candidate` handlers in `webrtcHandler.js` . Gated strictly: requests between peers >100px apart or in different offices are rejected with `PEER_OUT_OF_RANGE` . Media remains P2P.	**100% Complete** *(Verified)*
Task 7: Security & Performance	Prevent DB bottleneck on movement updates, sanitize coordinates, background sweep for stale peers.	In-memory spatial registry `officeManager.js` for sub-millisecond calculation.	**100% Complete** *(Verified)*

Task & Domain	Required Deliverables	What Was Implemented in Code	Status
		`positionSyncService.js` buffers updates and flushes to MongoDB every 2 seconds. Background cleaner sweeps stale peers.	
Task 8: Testing & GitHub	Comprehensive automated tests covering APIs, movement, proximity, signaling, error handling. Push to assigned branch.	27 out of 27 automated tests passing across 4 test suites using `node --test`. Legacy `test-socket.js` verified. Committed (`dd13151`) and pushed to `origin/saiprana v-backend`.	**100% Complete** *(Verified)*

3. Automated Test Evidence & Verification

A comprehensive automated test suite was built natively on Node.js:

```
text
▶ REST API Integration Tests
  ✓ GET / should return 200 and project info (48.82ms)
  ✓ GET /health should return 200 and status OK (5.19ms)
  ✓ POST /api/users without displayName should return 400 (16.98ms)
  ✓ POST /api/offices without name should return 400 (2.91ms)
  ✓ POST /api/positions with invalid userId should return 400 (2.57ms)
  ✓ POST with non-JSON Content-Type should return 415 (1.81ms)
  ✓ GET non-existing route should return 404 (2.46ms)
✓ REST API Integration Tests (88.27ms)

▶ OfficeManager - In-Memory State & Signaling Eligibility
  ✓ should allow a user to join an office and store state (1.96ms)
  ✓ should detect nearby peers on join and update bidirectional proximity sets (0.31ms)
  ✓ should keep peers in different offices completely isolated (0.43ms)
  ✓ should evaluate WebRTC signaling eligibility based on earshot radius (0.31ms)
  ✓ should clean up proximity and state when a peer leaves (0.43ms)
  ✓ should identify stale peers based on timeout (0.29ms)
✓ OfficeManager - In-Memory State & Signaling Eligibility (5.05ms)

▶ ProximityService - 2D Pixel Calculations
  ✓ should calculate exact Euclidean distance between two points (0.81ms)
  ✓ should return Infinity for invalid or missing coordinates (0.14ms)
  ✓ should correctly evaluate isWithinEarshot based on configured radius (10.25ms)
  ✓ should detect transitions: peer entering earshot (0.26ms)
  ✓ should detect transitions: peer leaving earshot (0.15ms)
  ✓ should detect transitions: peer maintaining earshot (0.15ms)
✓ ProximityService - 2D Pixel Calculations (13.14ms)

▶ Socket.io Proximity & WebRTC Signaling Integration Tests
  ✓ should handle office:join and provide existing peers list with earshot status (28.82ms)
  ✓ should broadcast avatar:moved to same office and maintain cross-office isolation
(80.11ms)
  ✓ should emit proximity:enter and proximity:leave transitions when moving relative to
100px earshot (9.29ms)
  ✓ should allow WebRTC offer/answer/ICE-candidate signaling when peers are within earshot
(9.09ms)
  ✓ should BLOCK WebRTC signaling and return webrtc:error when peers are outside earshot
(73.76ms)
  ✓ should BLOCK WebRTC signaling across different offices with DIFFERENT_OFFICE code
(7.16ms)
  ✓ should handle disconnect and emit user:left and proximity:leave to remaining peers
(6.27ms)
  ✓ should emit error event on invalid avatar:move coordinates (3.60ms)
✓ Socket.io Proximity & WebRTC Signaling Integration Tests (235.85ms)
```

```
ℹ tests 27 | ℹ suites 4 | ℹ pass 27 | ℹ fail 0 | ℹ duration_ms 1576.88
```

4. Established Event Contracts

Event Name	Direction	Payload Structure	Function / Purpose
`office:join`	Client → → Server	`{ officeId, userId, displayName, x, y }`	Joins office room, registers in-memory presence, updates DB socketId.
`office:joined`	Server → → Client	`{ status, officeId, peers: [...] }`	Acknowledges join and delivers existing peers with `isWithinEarshot` status.
`avatar:move`	Client → → Server	`{ x, y }`	Updates coordinates, triggers proximity transition check, queues DB save.
`avatar:moved`	Server → → Office Peers	`{ socketId, userId, x, y }`	Broadcasts position updates to peers in the same office room.
`proximity:enter`	Server → → Nearby Peers	`{ peerSocketId, peerUserId, distance }`	Triggered when distance ≤ 100≤ 100px. Triggers WebRTC connection setup.
`proximity:leave`	Server → → Nearby Peers	`{ peerSocketId, peerUserId, distance }`	Triggered when distance > 100> 100px or disconnect. Triggers audio stream teardown.
`webrtc:offer`	Peer ↔ ↔ Peer (Gated)	`{ to, offer }`	Forwarded only if peers are in the same office and within 100px earshot.
`webrtc:answer`	Peer ↔ ↔ Peer (Gated)	`{ to, answer }`	Forwarded only if peers are in the same office and within 100px earshot.
`webrtc:ice-candidate`	Peer ↔ ↔ Peer (Gated)	`{ to, candidate }`	Forwarded only if peers are in the same office and within 100px earshot.
`webrtc:error`	Server → → Sender	`{ status: 'error', code: 'PEER_OUT_OF_RANGE' }`	Emitted when signaling is attempted across offices or beyond 100px earshot.

5. GitHub Repository Verification

- **Repository:** `https://github.com/sathyaneelam/ProxiSpeak-Geospatial-Proximity-Audio-Ecosystem`
- **Assigned Branch:** `saipranav-backend`

- **Commit Hash:** `dd131515feac8afdf8f23d3127f87c1b356b5ea6`
- **Commit Message:** `feat(backend): implement proximity audio engine, WebRTC signaling, office presence, and automated test suite`
- **Remote Upstream:** `origin/saipranav-backend` (Verified up-to-date with 0 uncommitted changes)